definition, you pass the Frequent Flyer `status` parameter to the step definition. The actual name of the variable you use makes no difference; JBehave will pass any parameter values it finds to the annotated method in their order of appearance.

Steps often contain more than one piece of useful information, and you can pass any number of parameters into a step definition. For example, let's revisit the following step, which sets up the database with a flight for your test:

```
Given the flying distance between Sydney and Melbourne is 878 km
```

Here, there are three key fields you might want to pass to your step definition: the departure city, the destination city, and the distance. You could do this with a step definition like the following:

The step definition text, complete with variables ❶

```
@Given("the flying distance between $departure and $destination is
        $distance km")
public void defineTheFlyingDistanceForATrip(String departure,
                                            String destination,
                                            int distance) {
    // TODO: Set up the test database with this trip
}
```

The step definition method with matching parameters ❷

Here JBehave isolates the three fields you need and passes them to the step method. As mentioned earlier, the parameters extracted from the annotation ❶ are passed in to the method ❷ in their natural order; the parameter names are not important.

Notice how JBehave even converts the type for you, so you get the `distance` field directly in the form of an integer. JBehave will automatically convert numerical values, and with a little extra configuration, you can set it up to convert many other parameter types as well.

6.4.3 Sharing data between steps

When implementing step definitions, you'll frequently need to share data across multiple steps. For example, you might fetch a member account in one step, and then use this account in a subsequent step. You can do this with member variables, but this requires all of the step implementations to be in the same class or inherit from a common base class containing the shared member variables.

If you're using JBehave with Thucydides, a better alternative is to use the Thucydides session. Thucydides maintains a hash map for the life of a scenario that you can use to share data between steps, no matter what class they're implemented in. For example, you might retrieve a Frequent Flyer member from the database in a step like this:

```
@Given("a frequent flyer member called $name")
public void givenAFrequentFlyerMember(String name) {
    Member member = Members.findByName(name);
    Thucydides.getCurrentSession().put("member", member);
}
```

Fetch this member from the database.

Store it for later reference.